Understanding Large Language Models (LLMs)

A beginner-friendly guide to tokens, transformers, training, and RAG

1. What is an LLM?

A **Large Language Model (LLM)** is a machine-learning model trained on large amounts of data to process and generate language. Given a prompt, it predicts likely next tokens repeatedly, producing text that can answer questions, summarize, explain, translate, or help write code.

> **Important:** An LLM does not simply look up a prewritten answer. It generates a response from patterns learned during training and the context provided at inference time.

2. Tokens: the pieces a model processes

Text is converted into **tokens** before it enters the model. A token may be a whole word, part of a word, punctuation, or another text unit. Tokenization is not the same as splitting text into words or characters.

For example, a word such as “transformers” might be represented as one token or several smaller tokens, depending on the tokenizer. The model processes token IDs, not raw human-readable text.

3. Embeddings: turning tokens into vectors

Each token ID is mapped to a numerical vector called an **embedding**. These vectors let neural networks perform mathematical operations on token representations. Contextual representations change as tokens pass through the network, so the same word can be represented differently depending on surrounding text.

In document search, a separate embedding model can also turn a whole passage or question into a vector. Similarity between these vectors can help retrieve relevant passages. Those document embeddings are related to, but not necessarily the same as, an LLM's internal token embeddings.

4. Transformers and attention

Many modern LLMs use the **Transformer** architecture. A central component is **attention**, which helps the model combine information from different token positions. Attention assigns context-dependent weights to information, allowing the model to use relevant surrounding tokens when building representations.

> A simplified view of an LLM pipeline is: text → tokenization → token embeddings and positional information → repeated Transformer blocks → output scores for possible next tokens.

5. How an LLM generates text

At each generation step, the model produces scores for possible next tokens. A decoding method turns these scores into a choice. The selected token is added to the context, and the process repeats until a stopping condition is reached.

Temperature can affect how varied token selection is: lower settings tend to be more predictable, while higher settings tend to allow more variation. The exact effect depends on the model and decoding setup.

6. Training, inference, and fine-tuning

Pretraining: The model learns general patterns from a large training corpus, commonly by predicting missing or next tokens.

Inference: The trained model is used to produce outputs for a prompt. This is what happens when you ask an LLM a question.

Fine-tuning and alignment: Additional training stages can adapt a model to particular tasks, formats, or preferred response behavior.

7. What LLMs can and cannot do

- They can summarize, explain concepts, draft text, transform formats, and assist with code.
- They can make mistakes or produce confident-sounding statements that are not supported by evidence.
- Their answers depend on the prompt, available context, model capabilities, and training.
- They do not automatically know the current state of the world or the contents of your private files unless that information is supplied through an appropriate tool or context.

8. What is RAG (Retrieval-Augmented Generation)?

RAG combines information retrieval with LLM generation. Instead of relying only on what the model learned during training, an application searches a collection of documents, selects relevant passages, and supplies them to the model as context for its answer.

A typical document-RAG pipeline

1. **Load:** Extract text and useful metadata from files such as PDFs.
2. **Chunk:** Split long text into smaller passages, often with some overlap.
3. **Embed:** Convert passages into vectors using an embedding model.
4. **Index:** Store vectors and metadata so they can be searched efficiently.
5. **Retrieve:** Embed a user's question and find relevant passages.
6. **Generate:** Give the retrieved passages and question to the LLM.
7. **Cite:** Show source details such as filename and page number when the metadata supports them.

> **Why this matters for our Local Document AI project:** the PDF loader and chunker we have built cover the first stages. Next we will add embeddings and retrieval, then connect a local language model. Keeping each chunk's source and page metadata will help us show where an answer came from.

9. Key terms at a glance

Token: A unit of text processed by the model.

Embedding: A numerical vector representing a token, passage, or query.

Transformer: A neural-network architecture used by many modern LLMs.

Attention: A mechanism for combining information from relevant positions in context.

Inference: Running a trained model to generate an output.

Hallucination: An output that presents unsupported or incorrect information as if it were true.

RAG: Retrieving relevant external information and providing it to an LLM during generation.

10. Quick self-check

1. What is the difference between a token and an embedding?
2. Why is attention useful in a Transformer?

3 Why can an LLM produce an incorrect answer confidently?

4 How does RAG help an LLM answer questions about a private PDF?

5 Why should a document chunk keep its filename and page number?

Study tip: Explain the pipeline aloud in your own words: documents → chunks → embeddings → retrieval → LLM answer with sources.